Agile & MSA 실습 가이드

무엇을 이해하고, 무엇은 몰라도 되는가?

1. 왜 Agile 과 MSA 를 함께 배우는가?

소프트웨어 개발에서 가장 어려운 문제 중 하나는 "요구사항이 개발 도중에 계속 바뀐다"는 것입니다. 예전에는 처음에 완벽한 설계도를 그려놓고 그 순서대로 개발하는 방식(폭포수 방식)을 많이 사용했지만, 이 방식은 중간에 요구사항이 바뀌면 이미 만든 부분을 통째로 다시 만들어야 하는 문제가 있었습니다. 이런 경험을 직접 해본 적이 없더라도, AI 서비스처럼 빠르게 변화하는 분야에서는 처음에 세운 계획이 끝까지 그대로 유지되는 경우가 오히려 드물다는 점을 떠올려보면 쉽게 이해될 것입니다.

Agile 방법론은 이 문제를 "한 번에 완벽하게 만들기"가 아니라 "작게 나눠서 자주 만들고, 피드백을 반영해 계속 고쳐나가기"로 해결합니다. 이번 실습에서 스프린트1과 스프린트2로 나눈 것이 바로 이 방식입니다.

그런데 Agile 방식으로 "자주 고치고 다시 배포"하려면, 코드 구조 자체도 그것이 가능하도록 짜여 있어야 합니다. 만약 모든 기능이 하나의 거대한 프로그램으로 얽혀 있다면, 결제 기능 하나만 고쳐도 회원 기능·과목 기능까지 전부 다시 배포해야 하고 오류 위험도 커집니다. MSA(마이크로서비스 아키텍처)는 기능을 user-service, course-service, payment-service처럼 독립된 서비스 단위로 쪼개어, 한 서비스만 고쳐서 배포해도 다른 서비스에 영향을 주지 않도록 만드는 구조입니다. 즉 MSA는 Agile이 요구하는 "빠르고 잦은 변경"을 기술적으로 뒷받침해주는 아키텍처입니다.

이번 실습에서 여러분에게 요구되는 능력은 화면을 예쁘게 만드는 것이 아닙니다. 기획서를 쓰듯이 "이 AI 서비스가 어떤 이해관계자(사용자, 강사•판매자, 운영자 등)에게 어떤 가치를 주는가?"를 먼저 정의하고, 그 가치를 이번에 배우는 기술과 연결할 수 있는 능력이 핵심입니다. 그 가치를 어떤 기능은 스프린트1에 넣고 어떤 기능은 스프린트2로 미룰지 우선순위로 나누는 것, 그리고 각 서비스가 어떤 API를 주고받아야 그 가치가 실제로 전달되는지 명세를 설계하는 것 자체가 이번 실습에서 가장 중요하게 다뤄야 할 기획 역량입니다.

또 한 가지 기억할 점은, 이번 실습이 다루는 것은 전통적인 소프트웨어 개발이 아니라 AI

특성을 가진 소프트웨어 개발이라는 것입니다. 전통적인 소프트웨어는 요구사항이 비교적 명확하고 안정적인 경우가 많지만, AI 서비스는 모델 성능•사용자 반응•데이터에 따라 결과와 요구사항이 계속 바뀌고, 새로운 모델이나 에이전트 도구가 끊임없이 실험되고 교체됩니다. 이런 불확실성과 잦은 변화가 AI 소프트웨어 개발의 본질적인 특성이기 때문에, 작게 나눠 자주 검증하는 Agile과 모듈 단위로 독립적으로 교체 가능한 MSA가 유난히 잘 맞는 조합입니다. 여러분이 AI 서비스 엔지니어 커리큘럼에서 이 수업을 듣는 이유도 바로 여기에 있습니다.

정리하면, Agile은 "어떻게 일할 것인가"에 대한 방법론이고, MSA는 그 일하는 방식을 가능하게 하는 "기술 구조"입니다. 이 둘은 AI 소프트웨어 개발의 불확실성과 특히 잘 맞기 때문에 세트로 함께 쓰이며, 이번 실습은 이해관계자 가치 정의 → 스프린트 구분 → API 명세 설계로 이어지는, 앞으로 여러분이 AI 서비스•에이전트를 개발할 때 실제로 마주하게 될 기획•개발 방식을 미리 경험해보는 과정입니다.

2. 실무에서는 이렇게 활용된다 (Agile·MSA 의 진화 흐름)

많은 기업이 처음부터 MSA로 시작하지 않습니다. 스타트업이나 신규 서비스는 보통 하나의 프로그램 안에 모든 기능을 넣은 모놀리식(Monolith) 구조로 빠르게 만들어 시장에 먼저 선보입니다. 초기에는 팀 규모도 작고 기능도 단순해서, 오히려 하나로 뭉쳐 있는 편이 개발 속도가 빠르기 때문입니다.

그런데 서비스가 성장해 사용자와 기능이 늘어나면 문제가 생깁니다. 회원•결제•추천 기능이 하나의 코드에 얽혀 있으면 결제 로직 하나를 고치기 위해 전체 프로그램을 다시 테스트하고 배포해야 하고, 팀원이 늘어날수록 같은 코드베이스를 서로 건드리다 충돌도 잦아집니다. 이 시점에서 많은 회사가 기능 단위로 서비스를 쪼개는 MSA로 전환합니다. 특히 개인 사용자별로 특화된 서비스를 제공하는 엔터프라이즈 유형 중에서 Netflix, Amazon, 배달의 민족처럼 트래픽과 조직 규모가 큰 서비스일수록 MSA를 채택하는 이유가 여기에 있습니다.

MSA로 전환하면 배포와 확장은 자유로워지지만, 그 대가로 서비스 간 통신 관리, 데이터 일관성 유지, 운영 복잡도 증가라는 새로운 부담이 생깁니다. 그래서 실무에서는 "무조건 잘게 쪼갠다"가 아니라, 조직 규모와 서비스의 변경 빈도를 보고 얼마나 나눌지를 판단합니다. 최근에는 지나치게 잘게 쪼갠 MSA의 운영 부담을 줄이기 위해 관련 기능을 조금 더 큰 단위로 묶는 모듈러 모놀리식(Modular Monolith) 같은 절충안도 함께 논의되고 있습니다.

여러분이 이번 실습에서 다루는 것은 이미 MSA로 나뉜 상태의 서비스입니다. 하지만 실무에서는 이 구조에 도달하기까지 "왜 나눴는지", "얼마나 나눌지"를 판단하는 과정이 있었다는 것을 기억해 두면, 앞으로 실제 프로젝트에서 아키텍처를 설계할 때 훨씬 균형 잡힌 시각을 가질 수 있습니다.

3. 시작하기 전에 꼭 알아야 할 것

이번 실습의 목표는 MSA 인프라(Eureka, API Gateway, 인증 서버, Kafka)를 처음부터 설계하고 구현하는 능력이 아닙니다. 이미 만들어진 MSA 구조를 실무에서 신입 개발자가 그러하듯 "가져다 쓰는" 능력을 기르는 것이 목표입니다.

실제 회사에서도 신입 개발자가 입사 첫 주에 인프라팀이 구축한 인증 서버나 서비스 디스커버리 설정을 한 줄씩 다 이해하고 시작하지는 않습니다. 정해진 API 명세를 보고 그에 맞춰 화면을 만드는 것이 실제 협업 방식입니다.

제공된 백엔드 코드를 한 줄씩 읽으며 이해하려는 시도는 이번 실습 범위를 벗어난 것이며, 이해가 안 되는 것이 정상입니다.

4. 이해해야 할 것 vs 몰라도 되는 것

구성요소	알아야 할 것	몰라도 되는 것
user-service	회원가입·로그인 요청 형식, 응답으로 오는 토큰	비밀번호 암호화 로직, DB 내부 구조
course-service	상품(과목) 등록·조회 API 형식₩	JPA 매핑, 내부 쿼리
enrollment-service	"신청하면 이벤트가 발생하고, 결제 후 자동으로 상태가 바뀐다"는 흐름	Kafka Producer/Consumer 코드
payment-service	결제 완료 호출 → 이후 enrollment 상태가 바뀐다는 결과	결제 처리 내부 로직
auth-server / api-gateway / eureka-server	로그인하면 토큰이 나오고, 그 토큰을 헤더에 넣으면 다른 API가 동작한다는 사용법	내부 설정, 서명 알고리즘, 서비스 등록 방식

이 표가 핵심입니다. "몰라도 되는 것" 칸에 있는 내용을 이해하려고 애쓰다가 막힌 것이라

면, 그건 여러분의 문제가 아니라 실습 범위를 벗어난 질문을 스스로에게 던진 것입니다.

5. 본인 아이디어로 치환하는 법 (도메인 매핑표)

템플릿은 온라인 교육 플랫폼(강사•과목•수강신청•결제)이지만, 개념 구조만 같으면 어떤 서비스로도 치환할 수 있습니다. 다음 예시는 B2B 또는 B2BC 형태로 이해 바랍니다. 참고로 우리는 기업에게 가치를 제공하는 것이고, 해당 기업이 B2C를 하는 것입니다. 개발자가 직접 B2C 업무를 해야 하는 경우는 제외 바랍니다.

템플릿 개념	우리 팀 아이디어 (예 : 쇼핑몰이라면)
강사	판매자
과목 등록	상품 등록
수강신청	장바구니 담기 / 주문
결제	결제
수강권한 노출	주문 확정 후 상품 접근 허용
(빈칸)	직접 채워보세요 →

팀별로 이 표를 먼저 채우고 시작하세요. 표가 채워지면, 여러분이 만들 화면과 호출할 API가 거의 자동으로 정해집니다.

6. 이 프로젝트 구조에서 Agile•MSA 가 실제로 어떻게 드러나는가

1번에서 설명한 원리가 이번 실습에서는 구체적으로 이렇게 구현되어 있습니다. 스프린트1(회원•과목•수강신청)에서 먼저 핵심 기능만 동작하는 최소 버전(MVP : Minimum Viable Product)을 완성하고, 스프린트2(결제•이벤트 연동)에서 기능을 점진적으로 확장합니다. 이때 결제 기능을 새로 추가해도 스프린트1에서 만든 회원•과목 서비스는 건드릴 필요가 없는데, 이는 각 기능이 독립된 서비스(MSA)로 분리되어 있기 때문에 가능한 일입니다.

> *Agile의 "점진적 확장"과 MSA의 "독립된 서비스 단위"가 서로 맞물려야만 스프린트를 이렇게 자연스럽게 나눌 수 있습니다. 여러분이 지금 겪고 있는 실습 구조 자체가 그 실제 사례입니다.*

스프린트(스크럼)를 나누는 기준

이사할 때를 떠올려보면 쉽습니다. 이사 당일 짐을 한 번에 다 옮기지 않고, 없으면 당장 생활이 안 되는 필수품(침대, 냉장고)부터 먼저 옮기고, 있으면 편하지만 없어도 며칠은 버틸 수 있는 물건(장식품, 여분 가구)은 나중에 옮깁니다. 스프린트를 나누는 기준도 동일합니다. "이 기능이 없으면 사용자에게 서비스로서 가치를 줄 수 없는가?"를 기준으로, 없으면 안 되는 핵심 기능(회원가입•상품•신청)은 스프린트1로, 있으면 더 좋지만 없어도 서비스 자체는 굴러가는 기능(결제 자동화, 이벤트 연동)은 스프린트2로 미룹니다.

스크럼을 통해 실제로 얻는 것

스프린트를 나누는 이유는 단순히 "일정을 쪼개기 위해서"가 아닙니다. 첫째, 스프린트1이 끝나면 완성을 기다리지 않고 바로 동작하는 결과물로 피드백을 받을 수 있습니다. 둘째, 설계가 잘못된 부분(예 : 회원 인증 흐름 자체의 문제)을 프로젝트 초반에 발견할 수 있어, 다 만든 뒤에야 문제를 발견하는 폭포수 방식의 위험을 줄여줍니다. 셋째, 스프린트1의 피드백을 보고 스프린트2의 계획을 바꿀 수 있는 유연성이 생깁니다. 예를 들어 스프린트1을 시연해보니 결제보다 검색 기능이 더 급하다는 것을 알게 되면, 스프린트2의 우선순위를 그때 조정하면 됩니다.

팀원 역할에 따라 개인 과제가 달라져도 되는가?

네, 오히려 그래야 정상입니다. 같은 요리를 만드는 팀이라도 한 사람은 재료를 준비하고, 한 사람은 조리를 하고, 한 사람은 플레이팅을 맡는 것처럼, 스크럼 팀 안에서도 기획(Pain Point·우선순위 정의), 백엔드(API 구현·명세 작성), 프론트엔드(화면 구현•API 연동) 등 역할에 따라 개인 과제의 내용은 달라지는 것이 맞습니다. 중요한 것은 각자의 결과물이 "같은 스프린트 목표"로 다시 모여야 한다는 점입니다. 서로 다른 산출물이 API 명세라는 공통 약속을 통해 연결되기 때문에, 백엔드 담당이 서비스를 만드는 동안 프론트엔드 담당은 Swagger 문서만 보고 화면을 미리 만들 수 있습니다. 이것이 바로 MSA가 팀 협업 측면에서도 유리한 이유입니다 : 서비스가 독립적으로 나뉘어 있기 때문에 팀원들도 서로를 기다리지 않고 병렬로 작업할 수 있습니다.

7. 조별 발표 기획서에 포함해야 할 내용

발표 기획서는 아래 여섯 가지 흐름으로 구성해 주세요. 각 항목은 앞 단계의 결과가 다음 단계의 근거가 되도록 순서대로 이어져야 합니다.

1**. 이해관계자 가치**(Pain Point) : 우리 팀이 정의한 이해관계자(사용자, 강사·판매자, 운영자 등)가 실제로 겪는 불편함이 무엇인지, 왜 그 문제가 중요한지를 먼저 명확히 정의합니다. (5번 도메인 매핑표에서 정한 이해관계자를 기준으로 작성)

2. **이를 해결하기 위한 AI 솔루션** : 정의한 Pain Point를 어떻게 해결할지, 우리 서비스가 제공하는 핵심 기능과 그 안에서 AI가 담당하는 역할을 설명합니다.

3. **스프린트 구분** : 정의한 솔루션 중 어떤 기능을 스프린트1(핵심 가치, MVP)에서 먼저 구현하고 어떤 기능을 스프린트2(확장 기능)로 미룰지 우선순위를 정하고, 그렇게 나눈 이유를 설명합니다. (6번에서 다룬 스프린트1·스프린트2 구조를 우리 팀 서비스에 맞춰 적용)

4. **아키텍처 구성도** : 제공된 MSA 구조(user-service, course-service, enrollment-service, payment-service, Eureka, API Gateway, 인증 서버, Kafka)를 우리 팀 서비스명으로 치환한 다이어그램입니다. 어떤 서비스가 무엇을 담당하는지, 서비스 간 호출•이벤트 흐름을 화살표로 표시합니다.

5. **API 명세** : 프론트엔드가 실제로 호출하는 엔드포인트 목록(Method, URL, Request/Response 예시)입니다. Swagger UI에서 확인한 내용을 정리하면 됩니다. (Swagger UI를 보고 실제로 프론트엔드와 연결하는 방법은 부록 참고)

6. **동작 화면 스냅샷** : 실제로 만든 화면에서 위 API가 호출되어 동작하는 장면을 캡처합니다. 요청 전/후 상태 변화가 드러나도록 구성해 주세요 (예 : 수강신청 전 → 결제 → 신청 후 화면 노출).

> *이 여섯 단계는 실무에서 쓰이는 서비스 기획서의 기본 구조와 동일합니다. "왜(Pain Point) → 무엇을(솔루션) → 어떻게 나눠서(스프린트) → 어떻게 구현(아키텍처•API) → 결과(화면)"의 순서를 지키면, 발표를 듣는 사람도 우리 팀의 논리를 자연스럽게 따라올 수 있습니다.*

8. 실습 진행 순서 (체크리스트)

1. 팀 아이디어 확정 → 위 도메인 매핑표 작성

2. 제공된 API 문서를 보고 필요한 엔드포인트만 리스트업 (전체 코드를 읽을 필요 없음)

3. Swagger UI에서 해당 엔드포인트를 직접 호출(Try it out)해보고 요청/응답 형식만 확인 (백엔드 코드는 보지 않아도 됨)

4. HTML/JS(fetch)로 화면을 만들고, 3번에서 확인한 엔드포인트를 연결

5. 로그인 후 받은 토큰을 저장해두고, 이후 모든 요청 헤더에 포함

6. 스프린트2 : 결제 API를 호출한 뒤, enrollment 상태가 바뀌는지 "결과"만 확인 (Kafka 내부 동작은 몰라도 됨)

9. 막혔을 때 질문하는 법

질문을 던질 때 "Eureka가 왜 이렇게 짜여 있나요?"처럼 인프라 내부를 묻지 마시고, "이 API에 이 요청을 보냈는데 왜 이런 응답이 오나요?"처럼 본인이 실제로 호출한 지점으로 좁혀서 질문해 주세요.

> *질문 범위를 좁히면 훨씬 빠르게 답을 얻을 수 있고, 야간 보강 시간에도 이 레벨의 질문을 우선적으로 다루겠습니다.*

부록 1. Swagger UI 로 프론트엔드·백엔드 연결하기

일부 팀은 백엔드 연결 방법을 몰라 목업(mock) 데이터만으로 화면을 만들고 실제 연결은 건너뛰었습니다. 이 부록은 Swagger UI만 보고도 프론트엔드에서 실제 API를 호출할 수 있도록 단계별로 안내합니다. 여기까지만 따라오면, 백엔드 코드를 몰라도 실제로 동작하는 화면을 만들 수 있습니다.

1단계. Swagger UI 구조 읽기

API Gateway 주소 뒤에 Swagger 문서 경로로 접속하면, 서비스별로 태그가 묶여 있고 그 아래 Method(GET/POST/PUT/DELETE)와 경로(Path)가 나열되어 있습니다. 각 항목을 펼치면 Parameters(요청에 필요한 값), Request body(POST/PUT일 때 보낼 JSON 형식), Responses(응답 형식과 예시)를 볼 수 있습니다. 이 세 가지가 프론트엔드 코드를 작성하는 데 필요한 전부입니다.

2단계. Try it out으로 먼저 직접 호출해보기

코드를 짜기 전에 Swagger UI의 "Try it out" 버튼을 눌러 값을 입력하고 Execute를 눌러보세요. 어떤 요청을 보내야 어떤 응답이 오는지 코드 없이 먼저 눈으로 확인할 수 있습니다. Execute 후 화면 하단에 나오는 curl 명령어를 복사해두면, 이후 코드로 옮길 때 그대로 참고할 수 있습니다.

3단계. 인증이 필요한 API는 토큰부터 발급

로그인 API를 Try it out으로 먼저 호출해 응답으로 오는 토큰을 복사합니다. Swagger UI 우

측 상단의 "Authorize" 버튼을 누르고 "Bearer 복사한토큰" 형식으로 입력하면, 이후 모든 요청에 인증 헤더가 자동으로 포함되어 인증이 필요한 API도 바로 테스트할 수 있습니다.

4단계. 확인한 내용을 그대로 프론트엔드 코드로 옮기기

Swagger에서 확인한 Method·경로·Request body를 fetch로 옮기면 되는데, 이 fetch 코드 자체는 "어딘가에 떠 있는" 것이 아니라 반드시 HTML의 <script> 태그 안에, 그것도 "버튼을 눌렀을 때" 실행되도록 이벤트 리스너 안에 넣어야 합니다. 로그인 화면 전체를 예로 들면 다음과 같습니다.

```
<!-- login.html -->
<input id="email" type="text" placeholder="이메일">
<input id="password" type="password" placeholder="비밀번호">
<button id="loginBtn">로그인</button>

<script>
  document.getElementById("loginBtn").addEventListener("click", () => {
    fetch("http://localhost:8080/api/users/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        email: document.getElementById("email").value,
        password: document.getElementById("password").value
      })
    })
      .then(res => res.json())
      .then(data => {
        sessionStorage.setItem("token", data.token); // 다음 화면에서도 쓰기 위해 저장
        location.href = "course-list.html"; // 로그인 성공 후 다음 화면으로 이동
      });
  });
</script>
```

핵심은 두 가지입니다. 첫째, fetch는 addEventListener("click", ...) 안에 있어야 "버튼을 눌렀을 때" 동작합니다. 둘째, 로그인에서 받은 토큰은 변수(const token)에만 담아두면 페이지를 이동하는 순간 사라지므로, 여러 화면에서 계속 쓰려면 sessionStorage 같은 브라우저 저장소에 저장해야 합니다.

다음 화면(course-list.html)에서는 페이지가 열리자마자 저장해둔 토큰을 꺼내 인증이 필요한 API를 호출합니다.

```
<!-- course-list.html -->
<ul id="courseList"></ul>

<script>
  const token = sessionStorage.getItem("token");

  fetch("http://localhost:8080/api/courses", {
    method: "GET",
```

```
    headers: { "Authorization": "Bearer " + token }
  })
    .then(res => res.json())
    .then(courses => {
      const list = document.getElementById("courseList");
      courses.forEach(c => {
        const li = document.createElement('li');
        li.textContent = c.title;
        list.appendChild(li);
      });
    });
</script>
```

즉 "로그인 화면의 스크립트"와 "과목 목록 화면의 스크립트"는 서로 다른 HTML 파일에 각각 들어가고, 그 사이를 sessionStorage에 저장한 토큰이 이어주는 구조입니다. 화면(HTML)마다 필요한 API를 이런 식으로 하나씩 연결해 나가면 됩니다.

5단계. 자주 겪는 오류 : CORS

프론트엔드에서 호출했는데 브라우저 콘솔에 CORS 오류가 뜬다면, 개별 서비스 포트(8081~8085)로 직접 호출했기 때문일 가능성이 큽니다. 반드시 API Gateway 주소(8080)로 호출해야 합니다. 호출 주소가 http://localhost:8080/... 형태인지부터 확인하세요.

6단계. 그래도 막히면

목업으로 넘어가기 전에 1~3단계(Swagger에서 Try it out으로 성공하는 요청 확인)까지는 반드시 해보세요. Swagger에서 이미 성공한 요청이라면, 프론트엔드 코드로 옮기는 것은 그 URL·헤더·body를 그대로 옮기는 것뿐입니다. 이 단계에서 막혔다면 9번 항목의 방법대로 구체적으로 질문해 주세요.

부록 2. 자주 묻는 질문 (Q&A)

지금까지 나온 질문과 답변을 모아두었습니다. 본문에서 이미 다룬 내용도 있지만, 헷갈렸던 부분만 빠르게 다시 찾아볼 수 있도록 여기에 정리했습니다.

Q. fetch로 REST API를 연결하는 게 맞나요? JSON 형식으로 주고받으니 REST API로 봐도 되나요?

A. 네, 맞습니다. REST API는 URL(자원)·HTTP Method(GET/POST/PUT/DELETE)·JSON 등으로 데이터를 주고받는 "설계 방식"이고, fetch는 그 REST API를 브라우저에서 호출하는 "도구"일 뿐입니다. 지금 코드처럼 URL로 자원(users, courses)을 지정하고 Method로 동작

을 구분하고 JSON으로 데이터를 주고받고 있다면 REST API 방식이 맞습니다. fetch 대신 axios나 Postman•curl을 써도 마찬가지로 REST API 호출입니다.

Q. 전체적인 흐름(실무에서 어떻게 사용되는지, 왜 배워야 하는지)이 궁금합니다. MSA 이후 모듈러 모놀리식도 배우나요?

A. 실무에서 MSA가 어떻게 활용되고 왜 필요한지는 2번 섹션 "실무에서는 이렇게 활용된다"에서 다루고 있으니 참고해 주세요. MSA가 유일한 정답은 아니며, 조직•서비스 규모에 따라 모놀리식•모듈러 모놀리식•MSA 중에서 선택하는 것이 실무의 일반적인 접근입니다. SKALA 과정은 풀스택 개발자 양성 과정이라기 보다는, AI 서비스 엔지니어 양성과정이어서 AI 특성에 잘 맞는 MSA에 초점을 두고 있습니다.

Q. 스크럼(스프린트)을 나누는 기준이 무엇인가요? 스크럼을 통해 얻는 것은 무엇인가요? 팀원 역할에 따라 개인 과제 내용이 달라져도 되나요?

A. 6번 섹션에 자세히 설명해 두었습니다. 간단히 정리하면, 스프린트를 나누는 기준은 "이 기능이 없으면 서비스로서 가치를 줄 수 없는가?"이고, 스크럼을 통해 얻는 것은 빠른 피드백•리스크 조기 발견•유연한 우선순위 조정입니다. 팀원 역할에 따라 개인 과제 내용이 달라지는 것은 자연스러운 일이며, 각자의 결과물이 API 명세를 통해 하나의 스프린트 목표로 다시 연결되기만 하면 됩니다.